

University of Isfahan
Computer Engineering Department

Fundamentals and Applications of AI

Project 3: Neon Hunt

Escape Agent Challenge - Final Report

Prepared By:

Amir Khedri **4023613024**

Saman Sheikhalishahi **4023613041**

Instructor:
Dr. Marzieh Hosseini

July 3, 2026

Contents

1	Evaluation Logic and Heuristics	2
1.1	Monster Avoidance & Contextual Claustrophobia	2
2	Minimax and Alpha-Beta Implementation	2
3	Loop Prevention: The Flood-Fill Toxicity Map	3
4	Test Results and Observations	3
5	Boss Level Strategy (Bonus Implementation)	4

1. Evaluation Logic and Heuristics

The core evaluation function is designed to balance aggressive progression toward the exit with robust survival instincts. Instead of relying on a flat linear reward system, the heuristic utilizes advanced mathematical scaling to guide the AI naturally.

Non-Linear Exit Pull

To prevent the AI from experiencing "heuristic plateaus" where multiple distant paths appear equally viable, we implemented a non-linear gravity pull toward the exit:

$$\text{Score} += \left(\frac{10000.0}{d_{\text{exit}} + 1} \right)$$

This ensures that the reward for moving closer to the exit scales exponentially, easily overpowering minor localized penalties such as narrow corridors.

1.1. Monster Avoidance & Contextual Claustrophobia

The evaluation function utilizes tiered threat zones based on `bfs_distance` (accounting for maze walls, not just Manhattan distance). Imminent death is massively penalized (-50,000) to act as an absolute hard boundary. Critical danger applies a -3,000 penalty to force evasive action without completely paralyzing the AI's ability to slip past the monster.

Additionally, the AI possesses "Contextual Claustrophobia." If the AI detects it is in a one-way tunnel (`routes` ≤ 1) and the exit is not immediately inside that tunnel, it applies a heavy -4,000 penalty, preventing it from trapping itself.

2. Minimax and Alpha-Beta Implementation

The decision-making core utilizes the Minimax algorithm enhanced with Alpha-Beta pruning to explore future game states efficiently.

Alpha-Beta Pruning Mechanics

The `alpha_beta` function perfectly mirrors the decision tree of the standard `minimax` function. However, by tracking the maximum guaranteed score for the maximizing player (`alpha`) and the minimum guaranteed score for the minimizing player (`beta`), it actively prunes branches where $\text{beta} \leq \text{alpha}$. This allows the AI to search deeper into the decision tree within the same time constraints, returning the exact identical optimum move as standard Minimax but with significantly fewer explored states.

3. Loop Prevention: The Flood-Fill Toxicity Map

One of the most significant challenges in grid-based AI is "Horizon Blindness," which causes the agent to pace back and forth endlessly when the fear of the monster perfectly balances the desire to reach the exit.

To permanently eradicate infinite loops, we implemented a global **Flood-Fill Toxicity Map** utilizing a short-term memory dictionary (`_visit_counts`).

Dynamic Toxicity Scaling

Every time the player officially steps on a coordinate, that specific cell is logged. When evaluating future moves, the AI applies a cumulative penalty:

$$\text{Score} -= (\text{visits} \times 500)$$

If the AI attempts to pace in a corner, the localized tiles rapidly become so mathematically "toxic" that the AI is forced to break its own fear boundaries and push past the monster to find fresh, unvisited corridors.

4. Test Results and Observations

The agent was rigorously tested across various map difficulties using a consistent Search Depth of 4.

- **Level 1 (Easy):** The AI mapped a perfect, unhesitating straight line to the exit, utilizing the non-linear pull to navigate the most optimal route.

- **Level 2 (Normal):** Early iterations suffered from corner-trapping when the AI became too scared to bypass the monster. With the implementation of the Toxicity Map, the AI confidently navigated dead-ends, realizing that standing still was mathematically worse than risking a close-quarters bypass.
- **Level 3 (Hard):** The Alpha-Beta pruning demonstrated its immense value here, allowing rapid execution even in open spaces with high branching factors.

5. Boss Level Strategy (Bonus Implementation)

To secure the Boss Bonus points, the AI required specialized logic to survive a highly aggressive, fast-moving adversary. The standard evaluation was insufficient, as it treated a loss in 2 steps the same as a loss in 10 steps.

Depth-Weighted Terminal States

We modified the heuristic to pass the `current_depth` into the evaluation function:

- **PLAYER_WIN:** Returns $100000 + (\text{depth} \times 1000)$
- **MONSTER_WIN:** Returns $-100000 - (\text{depth} \times 1000)$

This fundamentally alters the AI's survival instinct. If a loss is inevitable, the AI will always choose the path that delays death the longest (higher remaining depth). This stall tactic frequently causes the fast-moving Boss monster to get caught behind maze walls, giving the AI the exact window it needs to escape and secure victory on Level 4.